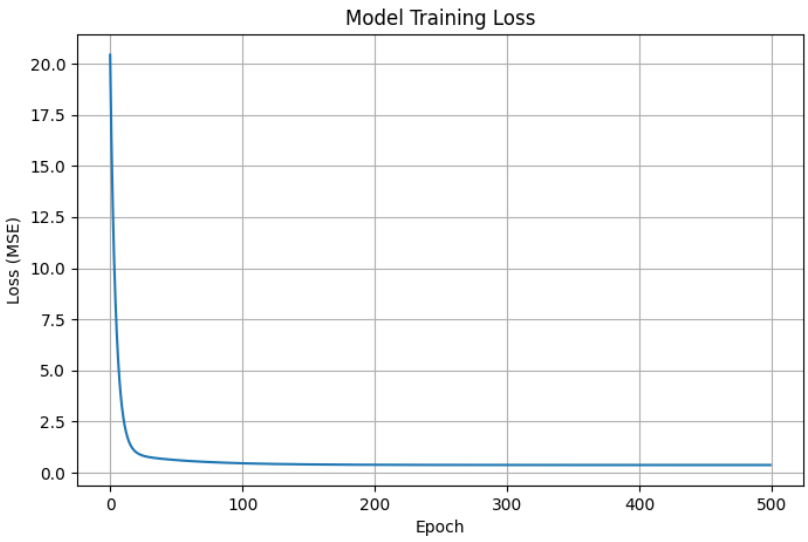
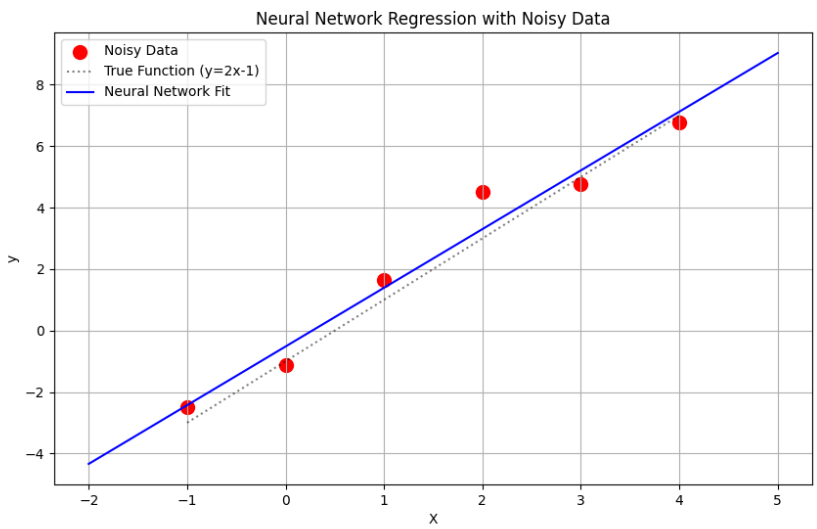
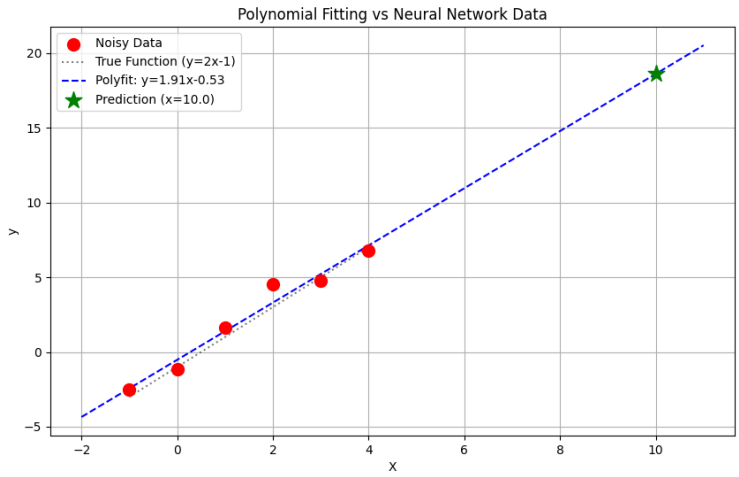
01_hello_nn.py 실행

```
(.venv) ➜  week1 git:(main) ✗ uv run python 01_hello_nn.py
warning: `VIRTUAL_ENV=/Users/hwang/Gwang/Class/aiclass/.venv` does not match the project environment path `/Users/
hwang/Gwang/Class/aiclass/AIandMLcourse/.venv` and will be ignored; use `--active` to target the active environmen
t instead
TensorFlow Version: 2.21.0

Training Data:
X: [-1.  0.  1.  2.  3.  4.]
y (clean): [-3. -1.  1.  3.  5.  7.]
y (noisy): [-2.50328585 -1.1382643   1.64768854  4.52302986  4.76584663  6.76586304]
/Users/hwang/Gwang/Class/aiclass/AIandMLcourse/.venv/lib/python3.12/site-packages/keras/src/layers/core/dense.py:1
06: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefe
r using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Starting training...
 Training finished!
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step

Prediction for x=10.0: 18.5852
Expected value: 19.0

Loss plot saved to outputs/training_loss.png
Model fit plot saved to outputs/model_fit.png

Learned Parameters:
Weight (w): 1.9104 (Expected: 2.0)
Bias (b): -0.5190 (Expected: -1.0)
Formula: y = 1.9104x + -0.5190
```





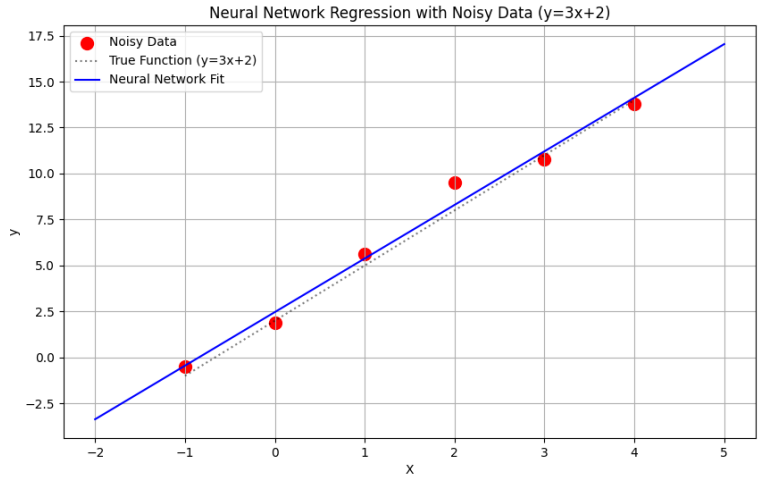
02_polynomial_fitting.py 실행

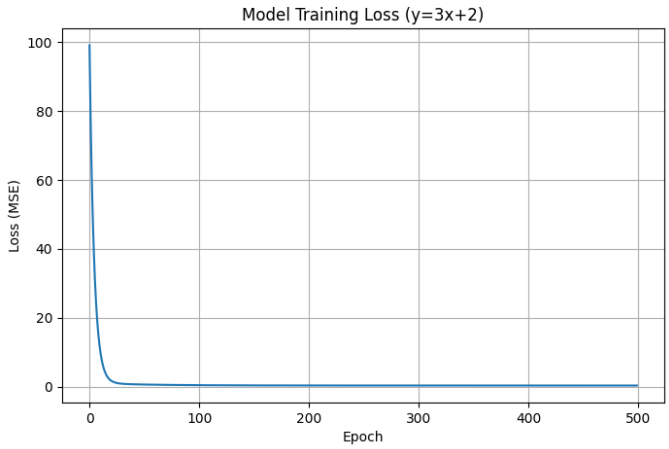



```
(.venv) ➜  aiclass cd AIandMLcourse/week1
(.venv) ➜  week1 git:(main) ✗ uv run python 02_polynomial_fitting.py
warning: `VIRTUAL_ENV=/Users/hwang/Gwang/Class/aiclass/.venv` does not match the project environment path `/Users/
hwang/Gwang/Class/aiclass/AIandMLcourse/.venv` and will be ignored; use `--active` to target the active environmen
t instead
Using CPython 3.12.12
Creating virtual environment at: /Users/hwang/Gwang/Class/aiclass/AIandMLcourse/.venv
Installed 46 packages in 552ms
==================================================
Numerical Methods vs Neural Networks
==================================================
Data:
X: [-1.  0.  1.  2.  3.  4.]
y (clean): [-3. -1.  1.  3.  5.  7.]
y (noisy): [-2.50328585 -1.1382643   1.64768854  4.52302986  4.76584663  6.76586304]
--------------------------------------------------

[Method 1] NumPy Polyfit (Least Squares)
Result: y = 1.9124x + -0.5251
Expected: y = 2.0000x + -1.0000
Prediction for x=10.0: 18.5987

[Method 2] SciPy Curve Fit (Optimization)
Result: y = 1.9124x + -0.5251
Prediction for x=10.0: 18.5987

Plot saved to outputs/02_numerical_fitting.png
--------------------------------------------------
Summary:
Neural Network (Previous): Iterative learning (Gradient Descent)
NumPy Polyfit: Analytical solution (Linear Algebra)
SciPy Curve Fit: Numerical optimization (Levenberg-Marquardt)
```

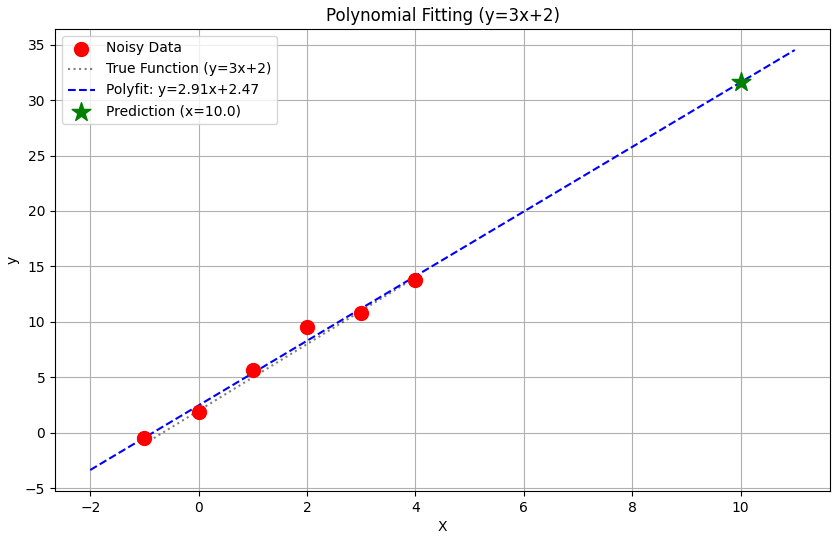
3x+2 학습 결과

```
➜  week1 git:(main) ✗  uv run python 01_hello_nn_y3x2.py
TensorFlow Version: 2.21.0

Training Data:
X: [-1.  0.  1.  2.  3.  4.]
y (clean): [-1.  2.  5.  8. 11. 14.]
y (noisy): [-0.50328585  1.8617357   5.64768854  9.52302986 10.76584663 13.76586304]
/Users/hwang/Gwang/Class/aiclass/AIandMLcourse/.venv/lib/python3.12/site-packages/ker
efer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Starting training...
Training finished!
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step

Prediction for x=10.0: 31.6136
Expected value: 32.0

Loss plot saved to outputs/training_loss_y3x2.png
Model fit plot saved to outputs/model_fit_y3x2.png

Learned Parameters:
Weight (w): 2.9145 (Expected: 3.0)
Bias (b): 2.4682 (Expected: 2.0)
Formula: y = 2.9145x + 2.4682
```

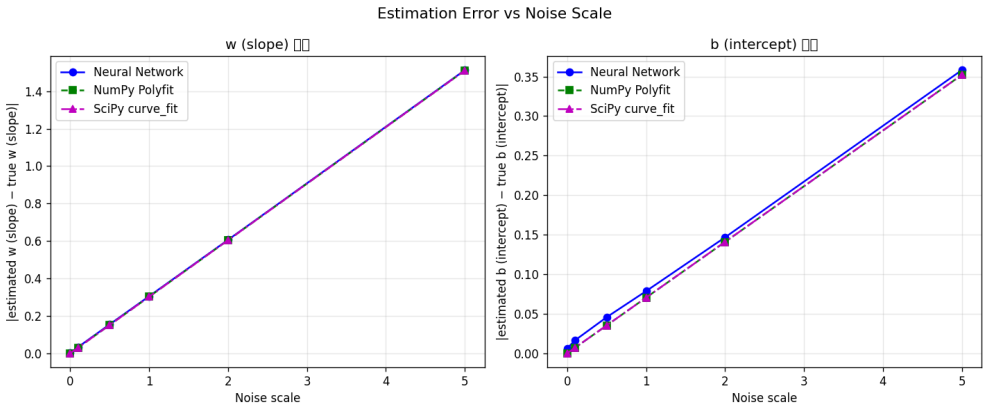


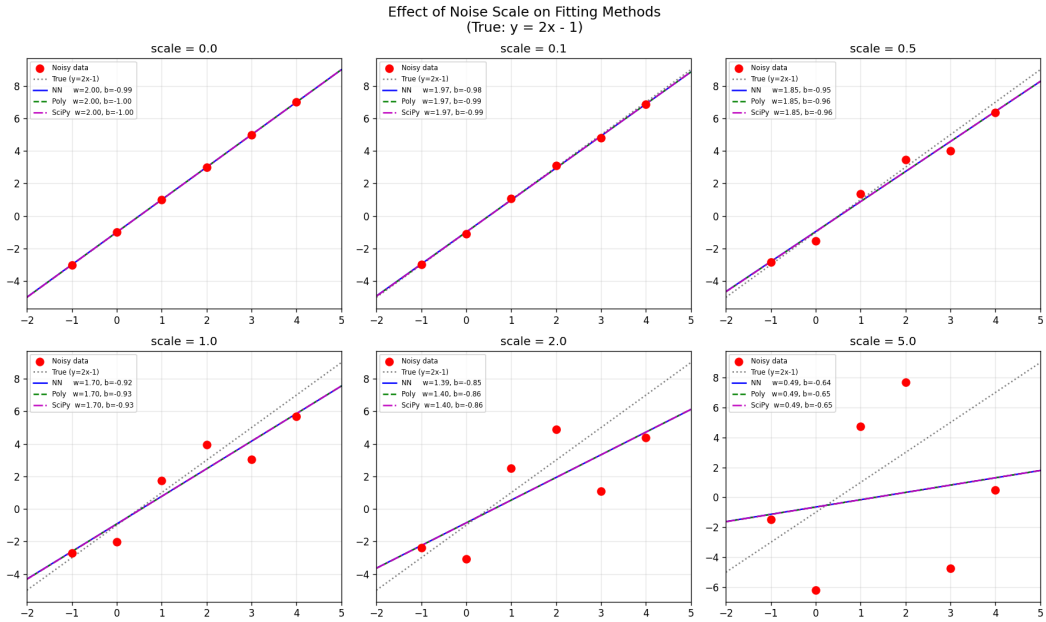



```
→  week1 git:(main) ✗  uv run python 02_polynomial_fitting_y3x2.py
==================================================
Numerical Methods vs Neural Networks
==================================================
Data:
X: [-1.  0.  1.  2.  3.  4.]
y (clean): [-1.  2.  5.  8. 11. 14.]
y (noisy): [-0.50328585  1.8617357   5.64768854  9.52302986 10.76584663 13.76586304]
--------------------------------------------------

[Method 1] NumPy Polyfit (Least Squares)
Result: y = 2.9124x + 2.4749
Expected: y = 3.0000x + 2.0000
Prediction for x=10.0: 31.5987
Expected value: 32.0

[Method 2] SciPy Curve Fit (Optimization)
Result: y = 2.9124x + 2.4749
Expected: y = 3.0000x + 2.0000
Prediction for x=10.0: 31.5987
Expected value: 32.0

Plot saved to outputs/02_numerical_fitting_y3x2.png
--------------------------------------------------
Summary:
Neural Network (01_hello_nn_y3x2.py): Iterative learning (Gradient Descent)
NumPy Polyfit: Analytical solution (Linear Algebra)
SciPy Curve Fit: Numerical optimization (Levenberg-Marquardt)
--------------------------------------------------
```


Polynomial Fitting (y=3x+2)
Noisy Data
True Function (y=3x+2)
Polyfit: y=2.91x+2.47
Prediction (x=10.0)
y
X

노이즈변경


Estimation Error vs Noise Scale
w (slope) □□
b (intercept) □□
Neural Network
NumPy Polyfit
SciPy curve_fit
|estimated w (slope) − true w (slope)|
|estimated b (intercept) − true b (intercept)|
Noise scale


Effect of Noise Scale on Fitting Methods
(True: y = 2x - 1)
scale = 0.0
scale = 0.1
scale = 0.5
scale = 1.0
scale = 2.0
scale = 5.0